Scalable Runtime Data Architectures for Low-Latency On-Device Speed-Limit Inference

Raphael Volz
Pforzheim University
raphael.volz@hs-pforzheim.de

February 26, 2026

Abstract

This paper evaluates runtime data architectures for low-latency, offline-capable smartphone speed-limit inference using country-scale OpenStreetMap (OSM) data. We identify four data processing scenarios that are compared to find an optimal query and update scenario along three typical query conditions. Hence, we identify the preferred operating point for practical deployment for on-device speed-limit inference as required for Intelligent speed assistance (ISA) applications that are mandatory for new vehicles in the EU since 2022 and open the way for a smartphone and open data based retrofitting approach for vehicles that are already in traffic. Thereby we can strongly support the EU traffic safety vision of reduced crash severity.

Keywords: Open Data OpenStreetMap map matching mobile GIS offline-first spatial indexing file-based preprocessing

1 Introduction

Excessive speed remains a major factor in crash severity, making reliable speed-limit assistance a core road-safety problem. For intelligent speed assistance (ISA), this creates a practical requirement for map context retrieval that remains accurate and low-latency under storage, memory, and connectivity constraints.

European regulation has made ISA deployment operationally mandatory. Article 6 of Regulation (EU) 2019/2144 requires ISA in new motor vehicles of categories M and N [1], and Delegated Regulation (EU) 2021/1958 specifies detailed ISA performance and test rules [2]. The rollout is phased: the requirement applies to new vehicle types from July 2022 and to all newly registered cars, vans, trucks, and buses from July 2024 [3]. The European Commission estimates that the broader General Safety Regulation package can prevent approximately 25,000 deaths and at least 140,000 serious injuries in the EU by 2038 [3].

A large legacy-vehicle gap remains. Eurostat reports that EU passenger-car stock exceeded 260 million vehicles in 2024 [4], while ACEA reports 10.6 million new EU car registrations in 2024 [5]. Using registration year as a coarse proxy, even an optimistic post-2024 assumption still leaves roughly 249 million pre-2024 vehicles (about 96% of the fleet) as potential retrofit targets.

This gap motivates a broad deployment approach based on smartphone sensing and open geospatial data. Eurostat reports mobile-phone internet use of 89% in cities and 82% in rural areas in 2024 [6], and OpenStreetMap (OSM) offers large-scale, continuously updated road-network attributes relevant for speed-limit inference [7; 8].

Against this background, this study examines runtime data architecture as the primary systems bottleneck. We compare four scenarios built from the same national OSM extract: global country-scale indexes (S1), tiled file packs (S2), a single-file spatially indexed database (S3), and a spatially indexed database with additional tile-window prefiltering (S4). The objective is to quantify latency and update-operability trade-offs under both cloud-like and on-device conditions.

2 Related Work

Map matching has been studied extensively for GPS trajectories under measurement noise and sampling sparsity. A foundational survey by Quddus et al. [9] systematized geometric, topological, and probabilistic matching strategies and outlined scaling challenges that remain relevant for mobile deployment.

Probabilistic sequence models became dominant for noisy and sparse data. Newson and Krumm [10] introduced a Hidden Markov Model (HMM) formulation that combines emission and transition probabilities, while Lou et al. [11] proposed ST-matching to jointly model spatial and temporal consistency for low-sampling trajectories. Real-time variants for streaming probe data were then explored in online HMM frameworks [12; 13].

Subsequent work strengthened route-level inference in sparse regimes. Rahmani and Koutsopoulos [14] formulated path inference from sparse floating-car data on urban networks, and the Path Inference Filter by Hunter et al. [15] provided a model-based low-latency approach for probe vehicles. Hybrid designs combining shortest-path priors with trajectory evidence further improved low-frequency matching [16], and route-choice-aware HMM approaches improved robustness for noisy sparse traces [17]. Enhanced HMM models that explicitly incorporate topology and traffic constraints also reported improved accuracy and consistency on floating-car data [18].

These studies primarily optimize candidate selection and path inference given an available road graph. In contrast, our contribution focuses on the systems layer around map data itself: physically tiled, content-addressed assets with independent update delivery and bounded local IO at query time. This targets a complementary bottleneck that is critical for country-scale, offline-first mobile speed-limit inference.

Spatial access methods are central to this systems layer. The R-tree introduced by Guttman [19] established the foundational dynamic index for multi-dimensional range search and remains a standard basis for geographic candidate retrieval in database systems. Here, R-tree-based indexing is evaluated not as a new algorithmic contribution, but as an architectural instrument to control candidate fan-out and improve practical query latency at country scale.

3 Technical Vision

3.1 Purpose

YouSpeed exists to make intelligent speed assistance practical for vehicles already on the road, not only for newly manufactured cars. The product target is an offline-first smartphone assistant that infers local speed limits with low latency, high availability, and transparent data provenance. The project target is broader: to show, with reproducible evidence, that open geospatial data combined with robust runtime architecture can provide safety-relevant functionality at consumer scale.

This work is intentionally dual-track. One track is scientific and publication-oriented, where architecture decisions are benchmarked and documented under controlled conditions. The second track is product-oriented, where validated architecture choices are integrated into an iPhone app under real update, reliability, and operational constraints.

3.2 Strategic phases

The system evolution is explicitly phased:

1. **Open-data bootstrap:** initialize the platform from OpenStreetMap and rule-aware defaults for country-scale baseline coverage.
2. **Product stabilization:** prioritize fast offline inference, deterministic updates, and measurable reliability on consumer devices.
3. **Community enrichment:** add user-observed speed-sign signals once install base and observation density are sufficient.

Crowdsourcing is defined as an augmentation layer, not a replacement for OSM baseline data.

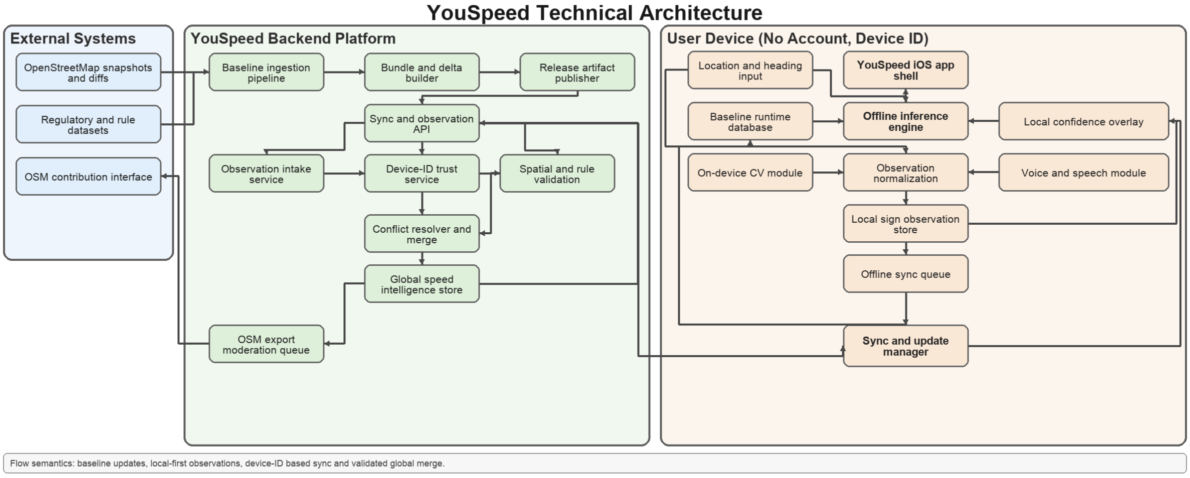

Figure 1: Technical architecture overview for baseline ingestion, on-device inference, local observation capture, and global merge/publish loops.

3.3 Crowdsourced speed intelligence

With sufficient adoption, two user-facing observation channels are introduced while driving. The first is on-device computer vision, where the smartphone camera detects speed-relevant signs when road-forward view is available. The second is voice input with speech recognition for hands-free reporting of sign changes, temporary restrictions, or map inconsistencies. Both channels generate candidate observations rather than immediate shared truth.

Each observation first updates a local device layer with confidence and decay semantics. Observations are then queued for deferred synchronization and promotion only after cross-device corroboration and validation.

3.4 Trust model without user accounts

YouSpeed is designed to operate without account authentication. Identity is pseudonymous and device-based. This lowers user friction and supports privacy goals, but shifts quality assurance into architecture: per-device source scoring, temporal/spatial consistency checks, minimum independent confirmations, and explicit conflict resolution.

A single-device report must not overwrite shared truth. Promotion to global data requires corroboration and consistency with geometry and legal plausibility constraints.

3.5 Layered data ownership and feedback loops

The data model is layered rather than monolithic:

- **External baseline layer:** imported from OSM and related official/open sources.
- **Internal global cache layer:** confirmed community observations maintained by YouSpeed.
- **Local device layer:** unconfirmed and recently confirmed observations relevant to one user.
- **Publication layer:** quality-gated candidates eligible for upstream contribution.

Confirmed findings follow two outbound paths: immediate use in the internal global cache and optional moderated feedback to OpenStreetMap via controlled export.

3.6 Synchronization as a core systems challenge

The central systems challenge is synchronization across truth sources with different latency and trust levels: local observations, globally confirmed internal data, external map updates, and periodic app bundles. Required capabilities include deterministic merge rules, versioned records, conflict precedence, rollback-safe activation, bandwidth-aware delta sync, offline accumulation, delayed upload, and eventual convergence without destructive overwrite.

3.7 App-side integration challenges

Computer vision and speech recognition are treated as separate subsystems with independent quality gates, failure modes, and runtime budgets. Integration happens through a shared observation contract so both channels emit comparable events for local storage, synchronization, and trust evaluation.

3.8 North-star outcome

The intended end state is a trusted open-data ISA retrofit platform that remains offline-capable and low-latency while improving continuously through external baselines and validated community observations. In that state, day-to-day user utility, auditable data operations, and reproducible research claims are aligned.

4 Methodology

4.1 Speed regulations in Germany

For passenger-car inference in Germany, this study uses the legal baseline defined in §3 StVO: 50 km/h in built-up areas and 100 km/h outside built-up areas unless traffic signs specify otherwise [20]. This rule is applied only when no explicit map speed value is available for a candidate road segment. The inference order is: user input (if present), then way-level `maxspeed` retrieval through `bbox`/`hybrid`/`polyline`, and finally legal-rule fallback, which requires city/rural classification via polygon containment.

To frame practical impact, Germany also applies escalating sanctions for speeding. Public guidance aligned with the current fine catalog reports, for example, +21 to +25 km/h in urban areas as EUR 115 plus one point in the driving register, and from +31 km/h as EUR 260 plus one point and a one-month driving ban [21]. These enforcement consequences motivate conservative fallback design in ISA research.

4.2 OpenStreetMap dataset potential for speed-limit inference

Tables 1 and 2 summarize the Germany OSM input used throughout this study. The dataset contains 7.96 million car-drivable ways and broad coverage of context objects (administrative boundaries, area contexts, and city place tags), which is sufficient to support both direct speed-limit lookup and fallback context inference.

The data also shows strong promise for practical speed-limit inference. More than 2.63 million drivable ways carry explicit `maxspeed` tagging, while high-volume road classes such as residential, secondary, and tertiary roads contribute substantial additional network coverage for rule-based inference when explicit values are absent. This combination of explicit speed tags and structured road semantics is a key enabler for scalable, offline-capable ISA research on open data.

The `maxspeed` counts in Table 2 refer to the base OSM key `maxspeed=*` on road ways (not derived keys such as `maxspeed:type`); they are distinct from area-context objects used for built-up-area inference.

Table 3 separates explicit and implicit speed-limit sources under German rule-aware inference. Beyond static snapshot downloads, OSM also supports incremental maintenance through replication diffs. In this study, daily diffs are used to quantify update feasibility and update workload for country-scale operation.

Table 1: Way counts by `highway` tag, with OSM-wiki interpretation and `maxspeed` tag occurrences [7; 8].

`highway` tag	Human-readable interpretation (OSM wiki)	`maxspeed` tagged (#)	Ways (#)
`service`	Access roads to premises, parking, and service areas.	144,112	3,928,743
`residential`	Roads primarily serving housing areas.	1,183,710	2,011,658
`secondary`	Secondary inter-settlement connectors in the network hierarchy.	425,993	506,931
`tertiary`	Tertiary connectors between local centers.	351,790	477,476
`unclassified`	Minor public through roads (not "unknown class").	152,237	417,457
`primary`	Primary roads with high network importance below trunk/motorway.	215,400	240,674
`living_street`	Traffic-calmed shared street (Wohnstraße / verkehrsberuhigter Bereich).	13,543	173,199
`motorway`	Controlled-access motorway (Autobahn-type) corridor.	69,075	70,200
`motorway_link`	Motorway ramps/connectors.	17,736	43,213
`trunk`	Major non-motorway long-distance routes.	31,594	33,284
`trunk_link`	Connector ramps between trunk and other classes.	10,486	21,538
`primary_link`	Connector ramps related to primary roads.	8,861	17,140
`secondary_link`	Connector ramps related to secondary roads.	8,262	13,854
`tertiary_link`	Connector ramps related to tertiary roads.	4,207	7,839
`road`	Temporary placeholder until precise class is surveyed.	14	275
Total	**All evaluated drivable `highway` categories in this study.**	**2,637,020**	**7,963,481**

Table 2: Germany dataset profile for evaluation.

Metric	Value
Input file size (`germany-latest.osm.pbf`)	4.70 GB
Car-drivable ways in evaluated classes	7,963,481
Ways with `maxspeed=*` tag	2,637,020
Ways with `zone:maxspeed=*` tag	172,116
Ways with explicit speed-limit tag (`maxspeed` or `zone:maxspeed`)	2,639,693
Administrative city-boundary candidates (`boundary=administrative`, `admin_level` 8/9)	44,621
Total area-context objects	129,459
Place objects tagged `city`	84

Table 3: Speed-limit provenance in the Germany dataset.

Category	Ways (#)	Share (%)
Explicit tagged limit (`maxspeed` or `zone:maxspeed`)	2,639,693	33.15
Implicit regulatory tag (`maxspeed:type`/`source:maxspeed` = DE urban/rural)	78,147	0.98
Default-rule fallback (context-based legal default)	5,245,641	65.87

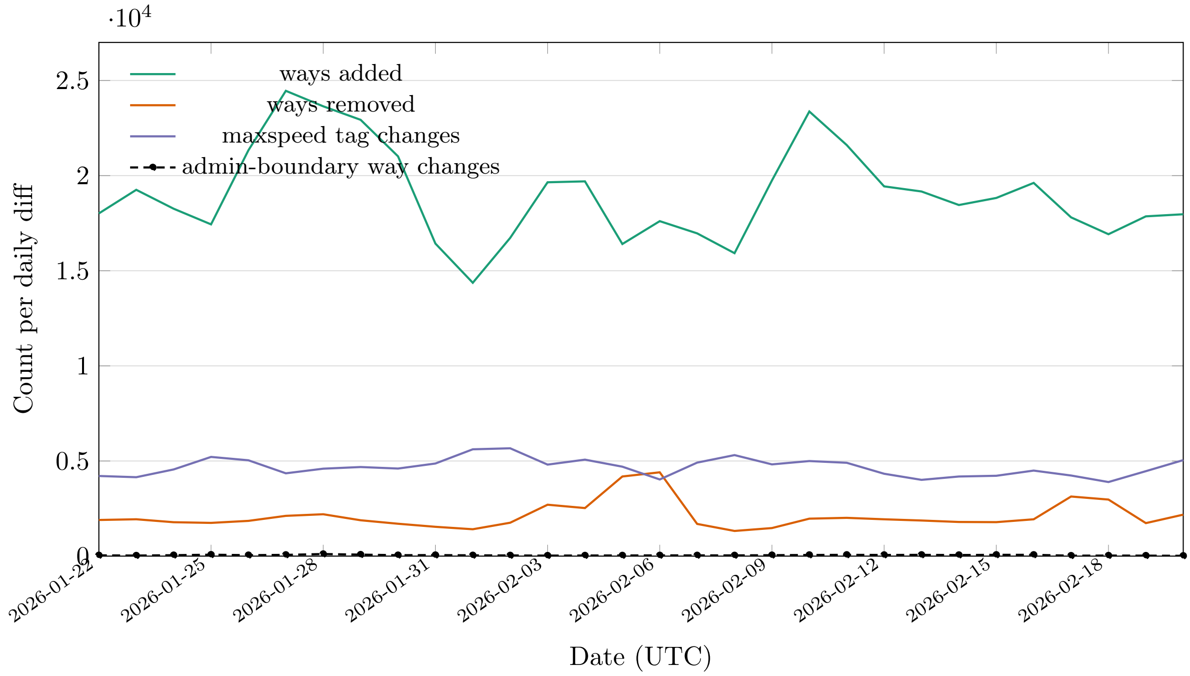

Figure 2: Daily update dynamics in the 30-day window: ways added, ways removed, `maxspeed`-tag changes, and administrative-boundary way changes per daily diff.

4.3 Data processing scenarios (S1–S4)

4.3.1 Shared preprocessing and scope

All four scenarios are generated from the same OSM extract (Geofabrik `germany-latest.osm.pbf`) [22]. Preprocessing follows a file-based parsing workflow [23] and does not rely on a separately deployed database server. The retained road network includes car-drivable `highway=*` categories (major roads, local roads, and connector links), ensuring that scenario differences arise from runtime data architecture and query execution rather than from differing road subsets.

4.4 Three query types

We use typical location-based queries to lookup speed limits for a vehicle in motion. All queries use the same score:

$$\text{score} = d + \lambda_h \cdot \Delta\theta + p_u$$

where d is the geometry term, λ_h heading weight (m/deg), $\Delta\theta$ bidirectional heading mismatch, and p_u optional unknown-highway penalty.

bbox query. d is haversine distance from the probe to the clamped point on a candidate bbox. It is the fastest approximation and is used as the coarse filter in every scenario.

polyline query. d is minimum point-to-segment distance across the full candidate geometry. This gives the highest geometric fidelity, especially on curved and elongated ways.

hybrid query. Candidates are first ranked with bbox distance. A limited top-N subset is then refined with polyline distance, combining near-polyline quality with lower compute cost.

4.4.1 Data processing architectures (S1-S4)

- S1 is the global-index baseline: country-scale index files are queried directly without prior spatial partitioning, which makes it simple but increases broad candidate scans and IO pressure for each lookup.

Table 4: Compared runtime scenarios (same input data, different runtime packaging/query path).

Scenario	Runtime artifact	Query characteristics
S1	Global country-scale index files	Broad candidate scans with comparatively high IO per query.
S2	Content-addressed spatial tile packs	Localized reads and independently refreshable regional partitions.
S3	Single-file spatially indexed embedded database	Direct spatial-index candidate retrieval with bounded fan-out.
S4	Spatially indexed embedded database with tile-window prefiltering	Two-stage filtering (tile prefilter then spatial-index intersection) for dense-area pruning.

- S2 introduces explicit spatial tiling and content-addressed tile assets. This localizes reads to relevant map partitions and supports selective data refresh at tile granularity.
- S3 uses a file-based embedded database with spatial indexing for candidate retrieval. S3 performs direct spatial-window search and then geometric ranking.
- S4 also uses a file-based embedded database but adds a tile-membership prefilter before spatial-index intersection to reduce candidate fan-out in dense urban regions.

Because S2–S4 runtime artifacts are file-based and immutable per build, they support horizontal cloud scaling through replication across stateless workers and object storage distribution, while also remaining suitable for autonomous offline execution on mobile devices.

4.5 Measurement scenarios

We evaluate two deployment modes:

- **Online (cloud-like) scenario:** host-side execution on a desktop-class environment, representing server-style continuous evaluation for online queries (measured without networking overhead).
- **Offline (on-device) scenario:** physical smartphone execution with country-scale runtime data stored in the mobile sandbox, representing autonomous offline operation.

Both modes are assessed the same way across the four runtime scenarios and the same three query types.

5 Evaluation

5.1 Experimental setup

The evaluated baseline was generated on 2026-02-23 from `germany-latest.osm.pbf`. Evaluation used a dual-platform setup: host-side cloud-like execution on a MacBook Pro (Mac16,5; Apple M4 Max, 16 cores; 48 GB RAM; macOS 15.5, Xcode 16.4) and offline execution on a physical iPhone 14 Pro (iPhone15,2; arm64e/A16 generation; iOS 26.2.1).

The benchmark protocol uses a fixed Berlin probe point $(52.5200, 13.4050)$, heading 90°, top-$k = 5$, and four measured query components (`bbox`, `hybrid`, `polyline`, `polycontainment`).

Runtime footprint is strongly architecture-dependent: S1 uses 7 runtime files (5.51 GB), S2 uses 109,333 files (5.10 GB), S3 uses a single file (2.60 GB), and S4 uses a single file (3.44 GB). A representative country-scale on-device run completed successfully on iPhone with one executed test and 48.331 s action elapsed time.

5.2 Benchmark results

Table 5 reports the joint online (cloud-like) and offline (device) latency matrix. Figure 3 orders the same data by scenario (S1–S4), each shown as adjacent online/offline columns.

Table 5: Joint online/offline benchmark matrix at the benchmark coordinate (ms).

Execution	Mode	S1 avg (ms)	S2 avg (ms)	S3 avg (ms)	S4 avg (ms)
online (cloud)	bbox	5451.24	172.21	64.74	166.26
online (cloud)	hybrid	10152.12	169.25	29.56	64.00
online (cloud)	polyline	10300.65	170.65	38.88	72.03
online (cloud)	polycontainment	1.51	1.07	0.94	0.44
offline (device)	bbox	4526.75	232.58	43.77	792.13
offline (device)	hybrid	2935.52	61.03	24.10	99.57
offline (device)	polyline	3122.36	76.20	39.16	78.54
offline (device)	polycontainment	3.14	0.81	0.74	2.79

To convert on-device latency into a raw location-update budget under worst-case per-fix operation (both way query and polygon containment query), this study uses:

$$f_{\text{cycle}} = \frac{1000}{t_{\text{maxspeed}} + t_{\text{polycontainment}}}, \qquad v_{\max} = 3.6 \cdot f_{\text{cycle}} \cdot \Delta s$$

where f_{cycle} is per-fix cycle throughput in Hz, t_{maxspeed} is way-query latency, $t_{\text{polycontainment}}$ is polygon containment latency, and Δs is distance between location fixes (here $\Delta s = 10\,\text{m}$). Table 6 reports per-scenario device throughput and the corresponding theoretical speed envelope without additional pipeline overheads.[1]

Table 6: On-device raw query throughput and theoretical speed envelope ($\Delta s = 10\,\text{m}$, worst-case per-fix latency $t_{\text{maxspeed}} + t_{\text{polycontainment}}$, no additional processing overhead).

Scenario	Mean latency (ms)	Mean throughput (Hz)	Best-mode $v_{\max}$ (km/h)	Worst-mode $v_{\max}$ (km/h)
S1	3531.35	0.28	12.2	7.9
S2	124.08	8.06	582.1	154.2
S3	36.42	27.46	1449.3	808.8
S4	326.20	3.07	442.7	45.3

Compared to S1, host-side speedups range from 31.7× (S2 bbox) to 343.4× (S3 hybrid). On device, the same overall ordering remains for hybrid and polyline ($S3 < S2 < S4 \ll S1$), while bbox in S4 regresses versus S3.

Based on current results, **S3** is selected as default on-device runtime: it delivers the lowest hybrid/polyline latency while keeping a single-file footprint smaller than S4.

For polygon containment (in-city/out-city classification), the measured city-context retrieval stage is 3.14 ms (S1), 0.81 ms (S2), 0.74 ms (S3), and 2.79 ms (S4). At the Berlin benchmark probe, all scenarios resolve *Mitte* as *inside-city* via explicit administrative polygon containment. The low millisecond-scale polycontainment latencies are expected because RTree prefiltering reduces candidate polygons to a very small set before exact ring containment checks.

[1] From a full-system perspective, location-fix cadence rather than query runtime is the dominant upper bound. Android compatibility requirements specify that handheld devices must report GPS location and GPS clock measurements at least once per second (≥1 Hz) [24]. The Android location framework also exposes millisecond-level request and minimum-update intervals [25], so higher cadences can be requested, but delivered rates remain device- and policy-dependent. With $\Delta s = 10\,\text{m}$, a guaranteed 1 Hz fix stream corresponds to 36 km/h; a requested 5 Hz stream corresponds to 180 km/h.

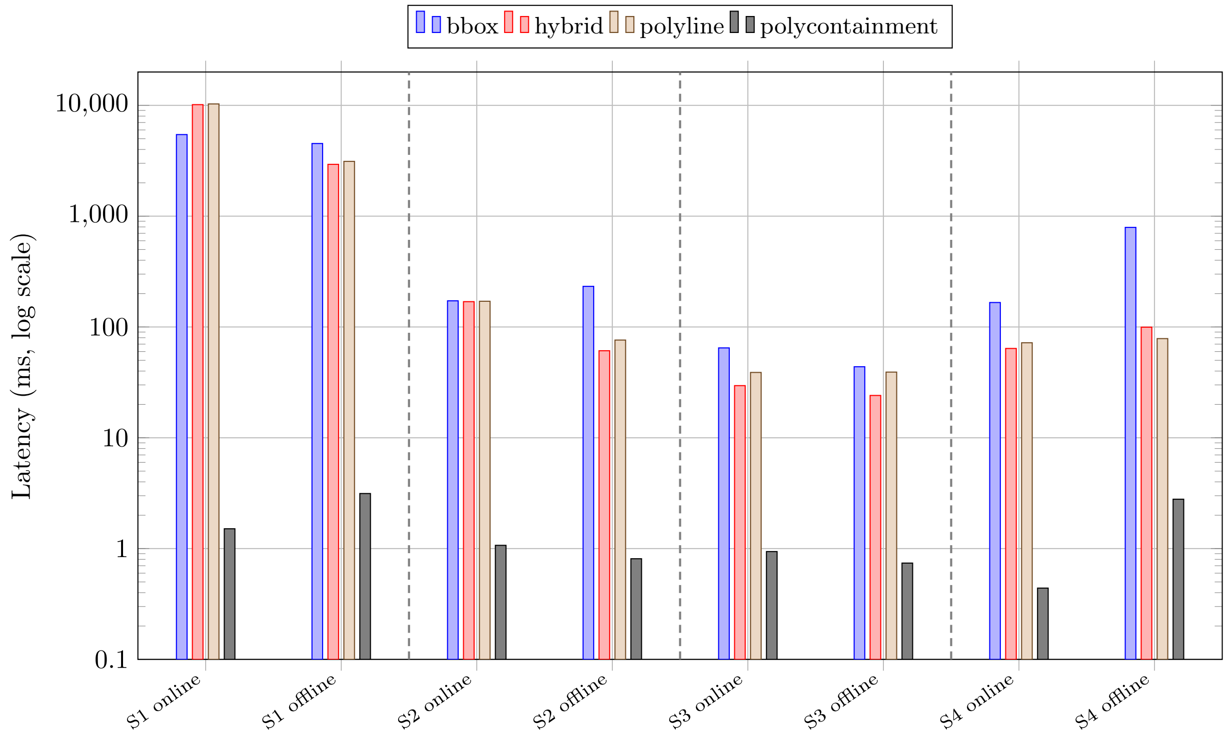

Figure 3: Scenario-first latency view derived from Table 5, including `polycontainment`. For each scenario (S1–S4), online and offline columns are adjacent, and dashed vertical lines delimit scenario groups.

6 Incremental Data Updates

Incremental update behavior was evaluated on a 30-day daily-diff window ending immediately before the baseline snapshot date (2026-01-22 to 2026-02-20). The analysis quantifies daily network churn, speed-tag churn, and scenario-specific refresh effort under a common observation period for the Germany subset of OSM.

Table 7: Daily-diff update workload summary (2026-01-22 to 2026-02-20, 30 days).

Metric	Total	Mean / day
Changed ways (+, −, modified)	1,829,171	60,972
`maxspeed`-related tag changes	139,999	4,667
Administrative-boundary way changes (`boundary=administrative`, level 8/9)	1,493	49.8
Invalidated S1 grid cells (S1 proxy)	201,355	6,712
Invalidated S2 tiles (S2 proxy)	80,536	2,685
S3 simulated patch runtime (S3)	30 719.38 ms	1023.98 ms
S4 simulated patch runtime (S4)	40 710.02 ms	1357.00 ms
Polygon-update S1 invalidated grid cells	15,151	505.03
Polygon-update S2 invalidated tiles	3,507	116.9
Polygon-update S3 simulated patch runtime	87.38 ms	2.91 ms
Polygon-update S4 simulated patch runtime	375.82 ms	12.53 ms

Across the measured window, S4 patching is 1.33× slower than S3 in mean daily patch runtime for maxspeed updates and 4.30× slower for polygon updates, while S2 invalidates substantially fewer partitions than S1. This indicates that tile partitioning improves refresh locality over global-index invalidation, and that S3 remains the stronger update/query compromise among database-centered variants.

To translate these architectural differences into deployment-relevant transfer times, payload transmission is represented as $T = (B_{\text{payload}} + B_{\text{overhead}})/R$, with B_{overhead} modeled as a separate 10% transport overhead term. For Europe, a practical throughput range is 104.3 Mbps to 217.6 Mbps for 5G downloads (spectrum-dependent) [26], with an EU-27 mobile average of 124 Mbps in 2025 [27]. Using measured median tile-pack size (47 199 B) and measured daily update volumes, indicative transfer times are: (i) 100 median tiles (4.72 MB payload): 0.19 s to 0.40 s; (ii) 1000 median tiles (47.2 MB): 1.91 s to 3.98 s; (iii) S2 mean daily invalidation volume (2685 median tiles, 126.73 MB): 5.13 s to 10.69 s; and (iv) S3 daily SQL patch transmission (gzip estimate, 5.07 MB): 0.21 s to 0.43 s. These results indicate that, in practical mobile operation, compact SQL patch delivery is typically faster than broad tile-bundle refresh for equivalent daily update coverage.

The daily benchmark is designed to be extended into a weekly longitudinal protocol with regular baseline checkpoints and intervening incremental updates. This will quantify whether reduced update effort is preserved over longer horizons while retaining low-latency query behavior.

7 Conclusion

Under a 50%/50% decision rule (equal weight on query latency and update effort), this study selects S3 as the preferred operating point. Query performance uses on-device mean latency, while update effort uses scenario-specific incremental-update proxies from Section 5 (partition invalidation for S1/S2, simulated patch runtime for S3/S4), normalized to a common $[0, 1]$ scale before weighting. The resulting ranking is $S3 \prec S4 \prec S2 \ll S1$ (lower is better).

Table 8: Scenario suitability matrix (++: best; +: suitable).

Scenario	Query time	Update time
S1	–	–
S2	+	+
S3	++	++
S4	+	+

Reproducibility

All preprocessing, scenario construction, and benchmark steps are parameterized and versioned in the accompanying research codebase available at https://github.com/volzinnovation/youspeed.de. The evaluation reported here is tied to the Germany snapshot dated 2026-02-23, with fixed probe location, heading, query modes, and scenario definitions stated in the methodology and evaluation sections.

References

[1] European Parliament and Council of the European Union. Regulation (eu) 2019/2144 on type-approval requirements for motor vehicles and their trailers, and systems, components and separate technical units intended for such vehicles. https://eur-lex.europa.eu/eli/reg/2019/2144/oj, 2019. Accessed: 2026-02-23.

[2] European Commission. Commission delegated regulation (eu) 2021/1958 supplementing regulation (eu) 2019/2144 by laying down detailed rules concerning intelligent speed assistance and driver drowsiness and attention warning systems. https://eur-lex.europa.eu/eli/reg_del/2021/1958/oj, 2021. Accessed: 2026-02-23.

[3] European Commission. New mandatory vehicle safety features from july 2024. https://single-market-economy.ec.europa.eu/news/new-mandatory-vehicle-safety-features-2024-07-05_en, 2024. Accessed: 2026-02-23.

[4] Eurostat. Passenger cars in the eu (statistics explained; data extracted in january 2026). https://ec.europa.eu/eurostat/statistics-explained/SEPDF/cache/25886.pdf, 2026. Accessed: 2026-02-23.

[5] European Automobile Manufacturers' Association (ACEA). Car registrations: +0.8% in 2024; battery electric 13.6% market share in december. https://www.acea.auto/pc-registrations/new-car-registrations-0-8-in-2024-battery-electric-13-6-market-share-in-december/, 2025. Accessed: 2026-02-23.

[6] Eurostat. Internet access in cities and rural areas. https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20241122-1, 2024. Accessed: 2026-02-23.

[7] OpenStreetMap Wiki Contributors. Key:highway. https://wiki.openstreetmap.org/wiki/Key:highway, 2026. Accessed: 2026-02-23.

[8] OpenStreetMap Wiki Contributors. Key:maxspeed. https://wiki.openstreetmap.org/wiki/Key:maxspeed, 2026. Accessed: 2026-02-23.

[9] Mohammed A. Quddus, Washington Y. Ochieng, and Robert B. Noland. Current map-matching algorithms for transport applications: State-of-the art and future research directions. *Transportation Research Part C: Emerging Technologies*, 15(5):312–328, 2007. doi: 10.1016/j.trc.2007.05.002. URL https://doi.org/10.1016/j.trc.2007.05.002.

[10] Paul Newson and John Krumm. Hidden markov map matching through noise and sparseness. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 336–343, 2009. doi: 10.1145/1653771.1653818. URL https://doi.org/10.1145/1653771.1653818.

[11] Yin Lou, Chengyang Zhang, Yu Zheng, Xing Xie, Wei Wang, and Yan Huang. Map-matching for low-sampling-rate gps trajectories. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 352–361, 2009. doi: 10.1145/1653771.1653820. URL https://doi.org/10.1145/1653771.1653820.

[12] C. Y. Goh, J. Dauwels, N. Mitrovic, M. T. Asif, A. Oran, and P. Jaillet. Online map-matching based on hidden markov model for real-time traffic sensing applications. In *2012 15th International IEEE Conference on Intelligent Transportation Systems*, pages 776–781, 2012. doi: 10.1109/ITSC.2012.6338627. URL https://doi.org/10.1109/ITSC.2012.6338627.

[13] Zhao-cheng He, She Xi-wei, Li-jian Zhuang, and Pei-lin Nie. On-line map-matching framework for floating car data with low sampling rate in urban road networks. *IET Intelligent Transport Systems*, 7(4):404–414, 2013. doi: 10.1049/iet-its.2011.0226. URL https://doi.org/10.1049/iet-its.2011.0226.

[14] Mahmood Rahmani and Haris N. Koutsopoulos. Path inference from sparse floating car data for urban networks. *Transportation Research Part C: Emerging Technologies*, 30:41–54, 2013. doi: 10.1016/j.trc.2013.02.002. URL https://doi.org/10.1016/j.trc.2013.02.002.

[15] Timothy Hunter, Pieter Abbeel, and Alexandre Bayen. The path inference filter: Model-based low-latency map matching of probe vehicle data. *IEEE Transactions on Intelligent Transportation Systems*, 15(2):507–529, 2014. doi: 10.1109/TITS.2013.2282352. URL https://doi.org/10.1109/TITS.2013.2282352.

[16] Mohammed Quddus and Simon Washington. Shortest path and vehicle trajectory aided map-matching for low frequency gps data. *Transportation Research Part C: Emerging Technologies*, 55:328–339, 2015. doi: 10.1016/j.trc.2015.02.017. URL https://doi.org/10.1016/j.trc.2015.02.017.

[17] George R. Jagadeesh and Thambipillai Srikanthan. Online map-matching of noisy and sparse location data with hidden markov and route choice models. *IEEE Transactions on Intelligent Transportation Systems*, 18(9):2423–2434, 2017. doi: 10.1109/TITS.2017.2647967. URL https://doi.org/10.1109/TITS.2017.2647967.

[18] Mingliang Che, Yingli Wang, Chi Zhang, and Xinliang Cao. An enhanced hidden markov map matching model for floating car data. *Sensors*, 18(6):1758, 2018. doi: 10.3390/s18061758. URL https://doi.org/10.3390/s18061758.

[19] Antonin Guttman. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, pages 47–57, 1984. doi: 10.1145/602259.602266. URL https://doi.org/10.1145/602259.602266.

[20] Bundesministerium der Justiz. Straßenverkehrs-ordnung (stvo), section 3 (geschwindigkeit). https://www.gesetze-im-internet.de/stvo_2013/__3.html, 2013. Accessed: 2026-02-23.

[21] ADAC. Bußgelder für geschwindigkeitsüberschreitungen (germany). https://www.adac.de/news/bussgeld-erhoeht-geschwindigkeitsueberschreitung/, 2025. Accessed: 2026-02-23.

[22] Geofabrik GmbH. Geofabrik openstreetmap data extracts – germany latest. https://download.geofabrik.de/europe/germany-latest.osm.pbf, 2026. Accessed: 2026-02-23.

[23] PyOsmium Contributors. Pyosmium documentation. https://docs.osmcode.org/pyosmium/latest/, 2026. Accessed: 2026-02-23.

[24] Android Open Source Project. Android 16 compatibility definition. https://source.android.com/docs/compatibility/16/android-16-cdd, 2026. Accessed: 2026-02-23.

[25] Android Open Source Project. Locationrequest api source documentation (frameworks/base). https://android.googlesource.com/platform/frameworks/base/+/refs/heads/main/location/java/android/location/LocationRequest.java, 2026. Accessed: 2026-02-23.

[26] Opensignal. Europe's mobile network experience 2024. https://www.opensignal.com/reports/2024/08/europe-mobile-network-experience, 2024. Accessed: 2026-02-23.

[27] European Commission. Impact assessment report accompanying the white paper "how to master europe's digital infrastructure needs?" (swd(2026) 13 final). https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52026SC0013, 2026. Accessed: 2026-02-23.